



AskMyDocs

RAG Document Q&A

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the RAG pipeline is wired.

Prepared 15 July 2026 · Author: Bhanu Prakash

o • The folder map

```
AskMyDocs/
├─ prisma/
│  ├─ schema.prisma      # User, Document, Chunk(embedding), Query
│  ├─ seed.ts            # demo user + ingests sample handbook
│  └─ sample-handbook.md # the demo document
├─ src/
│  ├─ app/
│  │  ├─ page.tsx        # landing
│  │  ├─ chat/           # the workspace (auth-gated)
│  │  │  └─ api/         # register, auth, documents, ask
│  │  └─ lib/            # parse, ocr, chunk, embed, retrieve, answer
│  │     └─ components/  # Workspace, DocumentLibrary, ChatPanel
│  │        └─ types/
│  └─ next.config.ts     # serverExternalPackages for native AI libs
└─ .env                  # DB url, secrets, Gemini key
```

The one rule: AI runs in exactly three places — `src/lib/embeddings.ts` (local embeddings), `src/lib/ocr.ts` (Gemini-vision OCR at ingest, only for scanned/image PDFs), and `src/app/api/ask/route.ts` (remote answer generation). Everything else goes through Prisma via the singleton client.

1 • Database & lib (the pipeline)

DATABASE BLUEPRINT

prisma/schema.prisma

PostgreSQL datasource + models: `User` ; `Document` (title, filename, mimeType, status, pageCount, chunkCount); `Chunk` (documentId, idx, page, content Text, `embedding` Json number[]); `Query` (logged Q&A with citations Json + model). Enum `DocStatus{PROCESSING,READY,FAILED}` .

PARSE • MULTI-FORMAT ROUTER

src/lib/parse.ts

Routes by file extension into a shared per-location `PageText[]` shape: **PDF** page-by-page (`pdf-parse`) with an OCR fallback when the text layer is empty; **Word** via `mammoth` ; **Excel/CSV** via `xlsx` (SheetJS) as **one unit per row** with headers baked in (`Sheet "X" • Row N`); **PowerPoint** by unzipping slide XML as **one unit per slide** (`Slide N`); **TXT/MD** as a single page.

★ OCR • VISION FALLBACK (NEW)

src/lib/ocr.ts

For image-based / scanned PDFs that yield no extractable text, sends the file to Gemini (`gemini-flash-latest`) for transcription and splits the result into per-page units on `=== PAGE n ===` markers. Reuses the same `GEMINI_API_KEY` as the answer path.

CHUNK

src/lib/chunk.ts

Structure-aware chunking (~450 chars, heading/paragraph aware, 80-char overlap, source location preserved).

★ EMBEDDINGS (LOCAL)

src/lib/embeddings.ts

`all-MiniLM-L6-v2` via `transformers.js`; mean-pool + L2-normalize; batch(32) + single; pipeline cached on `globalThis` .

INGEST ORCHESTRATOR

src/lib/ingest.ts

parse → chunk → embed → `createMany` → set READY/FAILED with counts.

RETRIEVE

src/lib/retrieve.ts + similarity.ts

Embed the query, cosine-score the user's READY chunks, return top-k (default 5).

ANSWER HELPERS

src/lib/answer.ts + constants.ts

Model names, `Citation` type, `toCitations`, `buildContext`, `SYSTEM_PROMPT`, `userPrompt`, `extractiveAnswer`; the shared `ANSWER_DELIMITER`.

AUTH · SESSION · DB

src/lib/{auth,session,prisma}.ts

NextAuth options; `getId()` server helper; singleton Prisma client.

2 · Routes, pages & components

Pages (src/app/)

FILE	PURPOSE
<code>layout.tsx</code>	Root layout, fonts, Providers (SessionProvider).
<code>page.tsx</code>	Landing; redirects to <code>/chat</code> if signed in.
<code>login/page.tsx</code>	Login / register form (demo creds pre-filled).
<code>chat/page.tsx</code>	Auth-gated workspace; server-loads the user's documents.

API routes (src/app/api/)

ENDPOINT	JOB
<code>register</code> · <code>auth/[...nextauth]</code>	Create account (bcrypt) · NextAuth handler.
<code>documents</code> (GET/POST)	List docs · upload a file — PDF, Word, Excel, PowerPoint, CSV or text (multipart) — and run the ingest pipeline.
<code>documents/[id]</code> (GET/DELETE)	Fetch one (ownership-checked) · delete (chunks cascade).
★ <code>ask</code> (POST)	Retrieve top-5 + stream a grounded answer with citations; log the Query.

Components (all client)

FILE	PURPOSE
<code>Providers</code> · <code>Navbar</code>	SessionProvider wrapper · top bar + sign-out.
<code>Workspace.tsx</code>	Holds docs list + selected scope; wires Library + ChatPanel.
<code>DocumentLibrary.tsx</code>	Upload UI (PDF/Office/CSV/text), doc list + status, scope select, delete.
<code>ChatPanel.tsx</code>	Chat UI: parses the streamed wire format, renders [n] badges + the Sources block + suggestions.

Parsing libraries (new): Word uses `mammoth`, Excel/CSV use `xlsx` (SheetJS), PowerPoint uses `jszip` to read slide XML — all pure-JS, no native binaries. **Native-package note:** `next.config.ts` marks `@xenova/transformers`, `onnxruntime-node`, `pdf-parse`, `pdfjs-dist`, `sharp` as `serverExternalPackages` so their native binaries aren't bundled.

3 • Commands

COMMAND	WHAT IT DOES
<code>npm install</code>	Install deps (+ <code>prisma generate</code>).
<code>npx prisma db push</code>	Create/sync the PostgreSQL tables.
<code>npm run db:seed</code>	Seed demo user + ingest the sample handbook (downloads the embedding model on first run).
<code>npm run dev</code>	Dev server → <code>localhost:3000</code> · demo <code>demo@askmydocs.app</code> / <code>demo1234</code> .

Gotchas: the schema and `.env` are **PostgreSQL**, but the committed migration is MySQL-dialect — use `prisma db push` to sync rather than `migrate` . The active answer path is **Gemini** (the Anthropic key is empty by default); the **same** `GEMINI_API_KEY` **now also powers scanned-PDF OCR**, so image-only PDFs need that key set. Embeddings always run locally, offline.